

WakeBuddy 설계서

1. 아키텍처 설계

1.1 설계 개요

WakeBuddy는 React Native, Expo, TypeScript를 기반으로 Android와 iOS에서 동작하는 모바일 앱으로 설계한다.

1.2 아키텍처 구조

WakeBuddy는 다음과 같은 계층 구조로 설계한다.

```
[사용자]
  ↓
[React Native / Expo App]
  ↓
[Screen Components]
  ├── SignUpScreen
  ├── LoginScreen
  ├── HomeScreen
  ├── MyAlarmScreen
  ├── AlarmFormScreen
  ├── FriendListScreen
  └── FriendAlarmScreen
  ↓
[Service Modules]
  ├── AuthService
  ├── AlarmService
  ├── FriendService
  ├── PermissionService
  └── NotificationService
  ↓
[External Infrastructure]
  ├── Firebase Authentication
  ├── Cloud Firestore
  └── Expo Notifications
```

1.3 계층별 역할

계층	구성 요소	역할
사용자 계층	사용자	앱을 통해 알람과 친구 기능을 사용하는 주체
화면 계층	Screen Components	사용자의 입력을 받고 처리 결과를 화면에 표시
서비스 계층	Service Modules	인증, 알람, 친구, 권한, 알림 관련 비즈니스 로직 처리
외부 인프라 계층	Firebase, Firestore, Expo Notifications	인증, 데이터 저장, 알림 예약 기능 제공

1.4 주요 구성 요소

React Native / Expo App

React Native와 Expo는 Android와 iOS에서 공통으로 동작하는 앱을 구현하기 위해 사용한다. 화면 구성, 사용자 입력 처리, 앱 내 이동 흐름은 React Native 기반 컴포넌트로 구현한다.

Screen Components

화면 컴포넌트는 사용자가 직접 상호작용하는 UI를 담당한다. 각 화면은 하나의 주요 책임을 갖도록 분리한다.

화면	역할
SignUpScreen	회원가입 화면
LoginScreen	로그인 화면
HomeScreen	주요 기능으로 이동하는 홈 화면
MyAlarmScreen	자신에게 설정된 알람 목록 조회
AlarmFormScreen	알람 생성 및 수정
FriendListScreen	친구 목록 조회, 친구 추가, 친구 삭제
FriendAlarmScreen	연결된 친구의 알람 조회, 생성, 수정, 삭제

Service Modules

서비스 모듈은 화면 컴포넌트와 외부 인프라 사이에서 비즈니스 로직을 담당한다. 화면 컴포넌트가 Firebase나 Firestore에 직접 접근하지 않고, 서비스 모듈을 통해 기능을 수행하도록 설계한다.

서비스	책임
AuthService	회원가입, 로그인, 로그아웃 처리
AlarmService	알람 조회, 생성, 수정, 삭제, 활성화/비활성화 처리
FriendService	친구 목록 조회, 친구 추가, 친구 삭제 처리
PermissionService	친구 관계와 알람 접근 권한 확인
NotificationService	알람 알림 예약, 취소, 재예약 처리

External Infrastructure

외부 인프라는 앱이 직접 구현하지 않고 외부 서비스를 활용하는 부분이다.

외부 인프라	역할
Firebase Authentication	사용자 회원가입, 로그인, 로그아웃 처리
Cloud Firestore	사용자, 친구 관계, 알람 데이터 저장
Expo Notifications	알람 시간에 맞춘 알림 예약 및 수신 처리

2. 품질 목표

요구사항 분석에서 도출된 비기능 요구사항을 설계 목표로 전환한다. WakeBuddy의 비기능 요구사항에는 Android/iOS 고려, 직관적인 화면 구성, 3초 이내 반영, 입력값 검증, 공통 앱 기능과 플랫폼별 알람 실행 기능 분리, 친구 관계 기준 조회, 사용자별 데이터 구분이 포함되어 있다.

비기능 요구사항	품질 목표	실제 설계 반영
Android와 iOS 환경을 모두 고려해야 한다.	플랫폼 호환성	React Native와 Expo로 공통 화면을 구현하고, 알람 기능은 NotificationService로 분리한다.
주요 화면은 단순하고 직관적이어야 한다.	사용성	SignUpScreen, LoginScreen, MyAlarmScreen, FriendListScreen 등 기능별 화면으로 분리한다.
알람 변경 결과는 3초 이내에 반영되어야 한다.	응답성	AlarmService 처리 후 화면 상태를 즉시 갱신 하여 알람 목록에 반영한다.
필수 입력값을 검증해야 한다.	안정성	AlarmForm과 AlarmService에서 제목, 시간, 대상 사용자 입력 여부를 검증한다.
공통 앱 기능과 플랫폼별 알람 실행 기능을 분리해야 한다.	유지보수성	화면, 서비스, 데이터 모델을 분리하고, 알람 실행은 NotificationService로 추상화한다.
친구 알람은 연결된 사용자 관계를 기준으로 조회되어야 한다.	보안성	PermissionService에서 친구 관계와 알람 접근 권한을 확인한다.
알람 데이터는 사용자별로 구분되어 저장되어야 한다.	무결성	Alarm 모델에 ownerId와 creatorId를 두어 알람 수신자와 생성자를 구분한다.

3. 상세 설계

3.1 데이터 모델 설계

WakeBuddy의 핵심 데이터 모델은 `User`, `Alarm`, `Friendship` 이다.

React Native와 TypeScript 환경이므로 `type` 또는 `interface` 를 사용하여 데이터 구조를 명확히 한다.

3.1.1 User 모델

속성	타입	설명
uid	string	Firebase Authentication에서 발급되는 사용자 고유 ID
email	string	사용자 이메일
displayName	string	사용자 이름
createdAt	Date	계정 생성 시각

```
export type User = {  
  uid: string;  
  email: string;  
  displayName: string;  
  createdAt: Date;  
};
```

3.1.2 Alarm 모델

속성	타입	설명
alarmId	string	알람 고유 ID
ownerId	string	알람을 받는 사용자 ID
creatorId	string	알람을 생성한 사용자 ID
title	string	알람 제목
time	Date	알람 시간
isActive	boolean	알람 활성화 여부
createdAt	Date	알람 생성 시각

속성	타입	설명
updatedAt	Date	알람 수정 시각

```
export type Alarm = {
  alarmId: string;
  ownerId: string;
  creatorId: string;
  title: string;
  time: Date;
  isActive: boolean;
  createdAt: Date;
  updatedAt: Date;
}
```

내 알람과 친구 알람을 하나의 Alarm 모델로 처리하기 위해 `ownerId` 와 `creatorId` 를 분리하였다.

```
내가 직접 만든 내 알람:
ownerId = 현재 사용자 uid
creatorId = 현재 사용자 uid

친구가 나에게 만든 알람:
ownerId = 현재 사용자 uid
creatorId = 친구 사용자 uid

내가 친구에게 만든 알람:
ownerId = 친구 사용자 uid
creatorId = 현재 사용자 uid
```

해당 구조를 사용해 내 알람과 친구 알람을 별도의 모델로 나누지 않고 하나의 알람 모델로 통합한다.

3.1.3 Friendship 모델

속성	타입	설명
friendshipId	string	친구 관계 고유 ID
userId	string	기준 사용자 ID
friendId	string	연결된 친구 사용자 ID
createdAt	Date	친구 관계 생성 시각

```
export type Friendship = {  
  friendshipId: string;  
  userId: string;  
  friendId: string;  
  createdAt: Date;  
};
```

친구는 별도의 사용자 종류가 아니라 사용자 간 연결 관계로 정의한다. 따라서 별도 사용자 클래스를 만들기보다 `Friendship` 모델을 통해 사용자 간 관계를 저장한다.

3.2 화면 및 컴포넌트 설계

WakeBuddy는 React Native 기반 앱이므로 화면 단위 컴포넌트를 중심으로 UI를 설계한다.

3.2.1 화면 설계

화면	역할
SignUpScreen	회원가입 기능 제공
LoginScreen	로그인 기능 제공
HomeScreen	주요 기능으로 이동하는 시작 화면
MyAlarmScreen	내 알람 목록 조회, 활성화/비활성화, 삭제 진입
AlarmFormScreen	알람 생성 및 수정 입력 화면
FriendListScreen	친구 목록 조회, 친구 추가, 친구 삭제 화면
FriendAlarmScreen	친구 알람 조회, 생성, 수정, 삭제 화면
SettingsScreen	로그아웃 및 앱 설정 화면

3.2.2 공통 컴포넌트 설계

컴포넌트	역할
AlarmItem	알람 목록에서 개별 알람을 표시
AlarmForm	알람 제목, 시간, 대상 사용자 입력
FriendItem	친구 목록에서 개별 친구를 표시
PrimaryButton	주요 동작 버튼
ErrorMessage	오류 메시지 표시
LoadingIndicator	데이터 로딩 상태 표시

3.2.3 화면 흐름

앱 실행
→ 로그인 여부 확인
→ 미로그인 상태: LoginScreen 또는 SignUpScreen
→ 로그인 상태: HomeScreen
→ 내 알람 관리: MyAlarmScreen → AlarmFormScreen
→ 친구 관리: FriendListScreen
→ 친구 알람 관리: FriendListScreen → FriendAlarmScreen → AlarmFormScreen

3.3 서비스 모듈 설계

화면 컴포넌트가 Firebase나 Firestore에 직접 접근하지 않도록 서비스 모듈을 분리한다.

서비스	책임
AuthService	회원가입, 로그인, 로그아웃 처리
AlarmService	알람 조회, 생성, 수정, 삭제, 활성화/비활성화 처리
FriendService	친구 목록 조회, 친구 추가, 친구 삭제 처리
PermissionService	친구 알람 권한 검증
NotificationService	알람 알림 예약, 취소, 재예약 처리

3.3.1 AuthService

메서드	설명
signUp(email, password, displayName)	새 사용자 계정을 생성한다.
login(email, password)	등록된 계정으로 로그인한다.
logout()	현재 로그인 상태를 종료한다.
getCurrentUser()	현재 로그인한 사용자 정보를 반환한다.

3.3.2 AlarmService

메서드	설명
getMyAlarms(userId)	사용자의 알람 목록을 조회한다.
getFriendAlarms(friendId)	연결된 친구의 알람 목록을 조회한다.
createAlarm(alarm)	새 알람을 생성한다.
updateAlarm(alarmId, data)	기존 알람 정보를 수정한다.
deleteAlarm(alarmId)	알람을 삭제한다.
toggleAlarm(alarmId, isActive)	알람을 활성화 또는 비활성화한다.

3.3.3 FriendService

메서드	설명
getFriends(userId)	사용자의 친구 목록을 조회한다.
addFriend(userId, friendId)	새로운 친구 관계를 생성한다.
removeFriend(friendshipId)	기존 친구 관계를 삭제한다.

3.3.4 PermissionService

메서드	설명
canCreateFriendAlarm(userId, friendId)	친구 알람 생성 가능 여부를 확인한다.
canViewAlarm(userId, alarm)	알람 조회 권한을 확인한다.
canEditAlarm(userId, alarm)	알람 수정 권한을 확인한다.
canDeleteAlarm(userId, alarm)	알람 삭제 권한을 확인한다.

알람의 조회, 수정, 삭제 권한은 `ownerId` 와 `creatorId` 를 기준으로 판단한다.

조회 가능 조건:

현재 사용자 uid == alarm.ownerId 또는 현재 사용자 uid == alarm.creatorId

수정 가능 조건:

현재 사용자 uid == alarm.ownerId 또는 현재 사용자 uid == alarm.creatorId

삭제 가능 조건:

현재 사용자 uid == alarm.ownerId 또는 현재 사용자 uid == alarm.creatorId

3.3.5 NotificationService

메서드	설명
scheduleAlarmNotification(alarm)	알람 시간에 맞춰 알람을 예약한다.
cancelAlarmNotification(alarmId)	예약된 알람을 취소한다.
rescheduleAlarmNotification(alarm)	수정된 알람 시간에 맞춰 알람을 다시 예약한다.

3.4 저장소 구조 설계

Cloud Firestore에는 사용자, 친구 관계, 알람 데이터를 분리하여 저장한다.

```
users
├── {uid}
│   ├── email
│   ├── displayName
│   └── createdAt
friendships
├── {friendshipId}
│   ├── userId
│   ├── friendId
│   └── createdAt
alarms
├── {alarmId}
│   ├── ownerId
│   ├── creatorId
│   ├── title
│   ├── time
│   ├── isActive
│   ├── createdAt
│   └── updatedAt
```

4. 주요 유스케이스 시퀀스 설계

4.1 로그인

사용자
→ LoginScreen
→ 이메일/비밀번호 입력
→ AuthService.login(email, password)
→ Firebase Authentication
→ 인증 결과 반환
→ HomeScreen 이동

4.2 내 알람 생성

사용자
→ MyAlarmScreen
→ AlarmFormScreen
→ 알람 제목 및 시간 입력
→ AlarmService.createAlarm()
→ Cloud Firestore에 알람 저장
→ NotificationService.scheduleAlarmNotification()
→ MyAlarmScreen 알람 목록 갱신

4.3 내 알람 활성화/비활성화

사용자
→ MyAlarmScreen
→ 알람 스위치 선택
→ AlarmService.toggleAlarm(alarmId, isActive)
→ Cloud Firestore에 활성화 상태 저장
→ isActive가 true이면 NotificationService.scheduleAlarmNotification()
→ isActive가 false이면 NotificationService.cancelAlarmNotification()
→ MyAlarmScreen 상태 갱신

4.4 친구 알람 생성

사용자

- FriendListScreen
- 연결된 친구 선택
- FriendAlarmScreen
- 친구 알람 생성 버튼 선택
- AlarmFormScreen
- 알람 제목 및 시간 입력
- `PermissionService.canCreateFriendAlarm(userId, friendId)`
- `AlarmService.createAlarm()`
- Cloud Firestore에 알람 저장
- `NotificationService.scheduleAlarmNotification()`
- FriendAlarmScreen 알람 목록 갱신

4.5 친구 알람 수정

사용자

- FriendAlarmScreen
- 알람 선택
- AlarmFormScreen
- 알람 제목 또는 시간 수정
- `PermissionService.canEditAlarm(userId, alarm)`
- `AlarmService.updateAlarm(alarmId, data)`
- Cloud Firestore에 수정 내용 저장
- `NotificationService.rescheduleAlarmNotification()`
- FriendAlarmScreen 알람 목록 갱신

4.6 친구 알람 삭제

사용자

- FriendAlarmScreen
- 알람 선택
- 삭제 버튼 선택
- `PermissionService.canDeleteAlarm(userId, alarm)`
- `AlarmService.deleteAlarm(alarmId)`
- Cloud Firestore에서 알람 삭제
- `NotificationService.cancelAlarmNotification(alarmId)`
- FriendAlarmScreen 알람 목록 갱신

5. 설계 원리 적용

5.1 추상화

WakeBuddy는 알람 실행 기능을 `NotificationService` 로 추상화한다.

화면 컴포넌트는 Android와 iOS의 알람 처리 차이를 직접 알 필요 없이

`scheduleAlarmNotification`, `cancelAlarmNotification`, `rescheduleAlarmNotification` 메서드만 호출한다.

이를 통해 플랫폼별 구현 세부사항을 숨기고, 공통 앱 기능과 플랫폼별 알람 기능을 분리할 수 있다.

5.2 캡슐화

알람 데이터의 생성, 수정, 삭제, 활성화/비활성화 처리는 `AlarmService` 를 통해 수행한다.

화면 컴포넌트는 Cloud Firestore에 직접 접근하지 않고 서비스 메서드를 호출한다.

이를 통해 데이터 저장 방식, 권한 검증, 알람 예약 로직을 서비스 내부에 숨길 수 있다.

5.3 모듈화

WakeBuddy는 인증, 알람, 친구, 권한, 알람 기능을 독립적인 모듈로 분리한다.

모듈	책임
AuthService	사용자 인증
AlarmService	알람 관리
FriendService	친구 관계 관리
PermissionService	권한 검증
NotificationService	알람 예약 및 취소

모듈화를 통해 기능 수정 시 영향 범위를 줄이고, 각 기능을 독립적으로 테스트할 수 있도록 한다.

5.4 결합도

화면 컴포넌트는 Firebase나 Firestore에 직접 의존하지 않고 서비스 모듈에 의존한다.

서비스 간에도 `userId`, `alarmId`, `friendId` 와 같은 단순 데이터를 전달하도록 설계하여 결합도를 낮춘다.

예를 들어 `FriendAlarmScreen` 은 Cloud Firestore에 직접 접근하지 않고 `AlarmService` 와 `PermissionService` 를 통해 친구 알람을 생성한다.

5.5 응집도

각 서비스는 하나의 목적에 집중하도록 설계한다.

`AuthService` 는 인증, `AlarmService` 는 알람, `FriendService` 는 친구 관계, `PermissionService` 는 권한 검증, `NotificationService` 는 알림 기능만 담당한다.

따라서 각 모듈은 관련 기능끼리 모여 있는 기능적 응집을 갖는다.

6. SOLID 원칙 적용

SOLID 원칙은 본 프로젝트의 전체 UI 구조보다는 서비스 모듈과 플랫폼별 알림 처리 구조에 선택적으로 적용한다.

(React Native는 컴포넌트 기반 프레임워크이므로 UI는 컴포넌트 중심으로 설계하고, TypeScript의 interface/service 구조를 활용하여 서비스 모듈에는 객체지향 설계 원칙을 적용한다.)

6.1 SRP: 단일 책임 원칙

각 서비스 모듈은 하나의 책임만 담당하도록 설계한다.

모듈	단일 책임
AuthService	사용자 인증
AlarmService	알람 데이터 관리
FriendService	친구 관계 관리
PermissionService	권한 검증
NotificationService	알림 예약 및 취소

6.2 OCP: 개방 폐쇄 원칙

알림 기능은 `NotificationService` 인터페이스를 기준으로 설계한다. 추후 Android와 iOS의 알림 구현 방식이 달라질 경우 컴포넌트는 수정하지 않고, 플랫폼별 구현체만 추가하거나 교체할 수 있도록 한다.

```
export interface NotificationService {
  scheduleAlarmNotification(alarm: Alarm): Promise<void>;
  cancelAlarmNotification(alarmId: string): Promise<void>;
  rescheduleAlarmNotification(alarm: Alarm): Promise<void>;
}
```

6.3 LSP: 리스코프 교체 원칙

`AndroidNotificationService` 와 `IOSNotificationService` 가 동일한 `NotificationService` 인터페이스를 구현한다면, 상위 모듈은 두 구현체 중 어떤 것을 사용해도 동일한 방식으로 알람 기능을 호출할 수 있다.

```
class AndroidNotificationService implements NotificationService {
    async scheduleAlarmNotification(alarm: Alarm): Promise<void> {
        // Android 알람 예약 처리
    }

    async cancelAlarmNotification(alarmId: string): Promise<void> {
        // Android 알람 취소 처리
    }

    async rescheduleAlarmNotification(alarm: Alarm): Promise<void> {
        // Android 알람 재예약 처리
    }
}

class IOSNotificationService implements NotificationService {
    async scheduleAlarmNotification(alarm: Alarm): Promise<void> {
        // iOS 알람 예약 처리
    }

    async cancelAlarmNotification(alarmId: string): Promise<void> {
        // iOS 알람 취소 처리
    }

    async rescheduleAlarmNotification(alarm: Alarm): Promise<void> {
        // iOS 알람 재예약 처리
    }
}
```



```
void> {
    // iOS 알림 재예약 처리
}
}
```

6.4 ISP: 인터페이스 분리 원칙

인증, 알림, 친구, 권한, 알림 기능을 하나의 거대한 인터페이스로 묶지 않고 각각 분리한다.

```
AuthService
AlarmService
FriendService
PermissionService
NotificationService
```

이를 통해 특정 화면이 필요하지 않은 기능에 의존하지 않도록 한다.

6.5 DIP: 의존관계 역전 원칙

화면 컴포넌트는 Firebase Authentication, Cloud Firestore 같은 구체 구현에 직접 의존하지 않고, `AuthService`, `AlarmService` 같은 서비스 추상화에 의존하도록 설계한다.

```
나쁜 구조:
LoginScreen → Firebase Authentication 직접 호출

개선된 구조:
LoginScreen → AuthService → Firebase Authentication
```

7. 요구사항 추적

요구사항 분석에서 도출된 기능이 설계 요소와 연결되는지 확인하기 위해 다음과 같이 추적한다.

요구사항	설계 반영
회원가입, 로그인, 로그아웃	AuthService, SignUpScreen, LoginScreen, SettingsScreen
내 알람 조회	AlarmService.getMyAlarms(), MyAlarmScreen
내 알람 생성	AlarmService.createAlarm(), AlarmFormScreen
내 알람 수정	AlarmService.updateAlarm(), AlarmFormScreen
내 알람 활성화/비활성화	AlarmService.toggleAlarm(), AlarmItem
내 알람 삭제	AlarmService.deleteAlarm(), MyAlarmScreen
친구 목록 조회	FriendService.getFriends(), FriendListScreen
친구 추가	FriendService.addFriend(), FriendListScreen
친구 삭제	FriendService.removeFriend(), FriendListScreen
친구 알람 조회	AlarmService.getFriendAlarms(), FriendAlarmScreen
친구 알람 생성	PermissionService.canCreateFriendAlarm(), AlarmService.createAlarm(), FriendAlarmScreen
친구 알람 수정	PermissionService.canEditAlarm(), AlarmService.updateAlarm(), FriendAlarmScreen
친구 알람 삭제	PermissionService.canDeleteAlarm(), AlarmService.deleteAlarm(), FriendAlarmScreen
알람 또는 알림 수신	NotificationService, Expo Notifications
Android/iOS 환경 고려	NotificationService 인터페이스 및 플랫폼별 구현체
입력값 검증	AlarmForm, AuthService, AlarmService
사용자별 데이터 구분	User.uid, Alarm.ownerId, Alarm.creatorId, Friendship.userId